



REPORTE DE ACTIVIDAD 10

Nombre del proyecto: File path traversal, traversal sequences stripped with superfluous URL-decode

Nivel: Apprentice

Nombre: Karla Vanessa Glz Espinoza

TIPO DE EXPLOTACIÓN

Path Traversal Bypass mediante Double URL Encoding / CWE-22: Improper Limitation of a Pathname to a Restricted Directory.

OBJETIVO DEL LABORATORIO

El objetivo de esta práctica es evadir los controles de seguridad que bloquean secuencias de evasión de directorios (path traversal) y sus variantes con codificación URL simple, explotando una vulnerabilidad de decodificación URL redundante (superflua) para lograr acceso al archivo protegido /etc/passwd.

EXPLICACIÓN TÉCNICA

Los atacantes comúnmente utilizan la codificación URL (URL encoding) para evadir filtros de seguridad. En la codificación URL simple, los caracteres como el punto (.) y la barra diagonal (/) se representan como %2e y %2f, respectivamente. Los servidores web modernos decodifican automáticamente estos valores antes de procesar la solicitud (OWASP Foundation, 2023).

Para protegerse, los desarrolladores suelen implementar validaciones que bloquean tanto el texto plano (../) como su representación codificada una sola vez (%2e%2e%2f). Sin embargo, una vulnerabilidad de "Double URL Encoding" (Doble Codificación URL) surge cuando la aplicación o el framework decodifican los datos de entrada de forma redundante (dos o más veces) después de haber ejecutado el control de seguridad (MITRE, 2023).

En un ataque de doble codificación, el atacante codifica el propio carácter de escape % como %25. De este modo, %2e se convierte en %252e. Cuando este payload pasa por el filtro de seguridad (WAF o validación de código), el filtro no reconoce la firma maliciosa y lo permite. Posteriormente, la rutina de decodificación superflua en el backend transforma el %252e de regreso a %2e, y luego a .., ensamblando el payload ../ final justo antes de ser enviado a la API del sistema de archivos (CISA, 2022).

PROCEDIMIENTO

Paso 1: Se inició el entorno virtual del laboratorio provisto por Web Security Academy y se configuró Burp Suite Community Edition para la intercepción de tráfico.

Paso 2: Se accedió a la página de descripción de un artículo en el navegador de Burp. En la pestaña de HTTP history, se localizó la petición encargada de renderizar la imagen del producto (GET /image?filename=...jpg). Se envió esta petición a la herramienta Repeater.

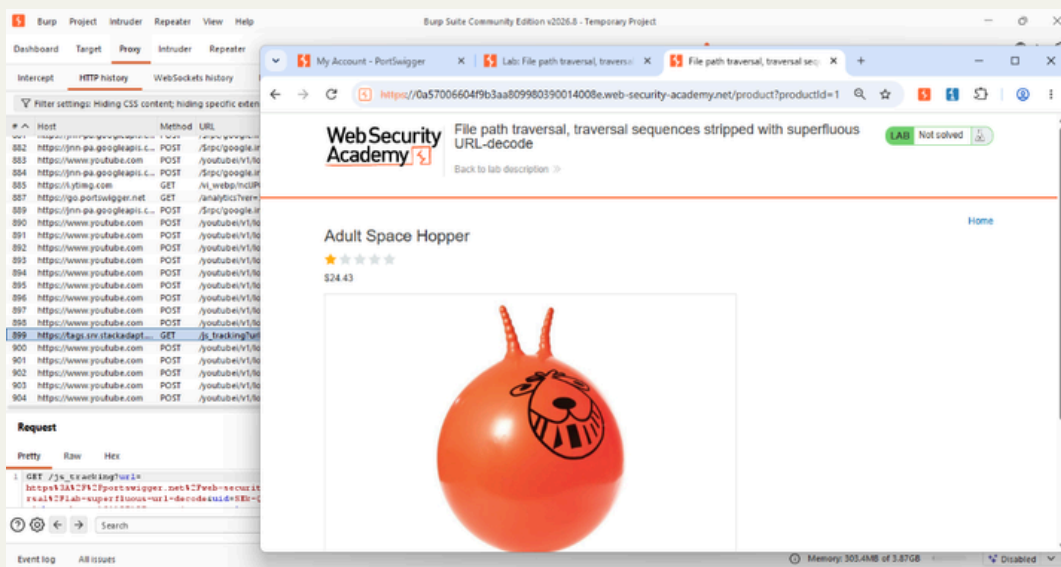


Figura 10.1 Pagina principal de la tienda en un apartado de “View details” de un producto.

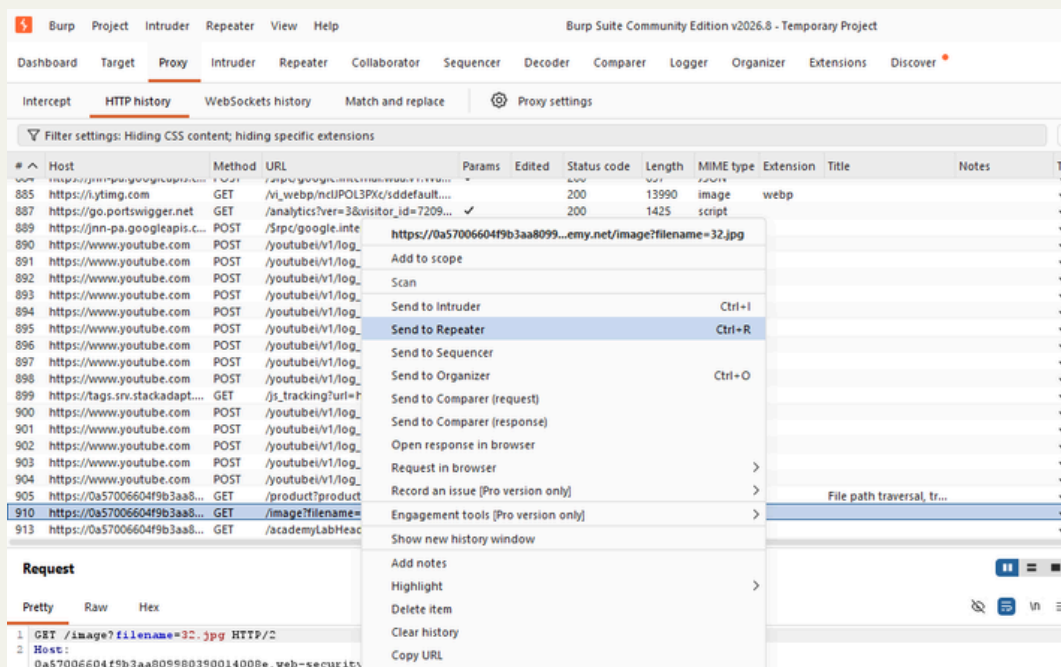


Figura 10.2 Se envió la petición a Repeater para su maipulación

Paso 3: Se observó que la aplicación bloqueaba exitosamente tanto secuencias relativas estándar (../) como codificaciones simples. Por ello, en el panel del Repeater, se modificó el parámetro filename inyectando el payload con doble codificación URL: %252e%252e%252f.

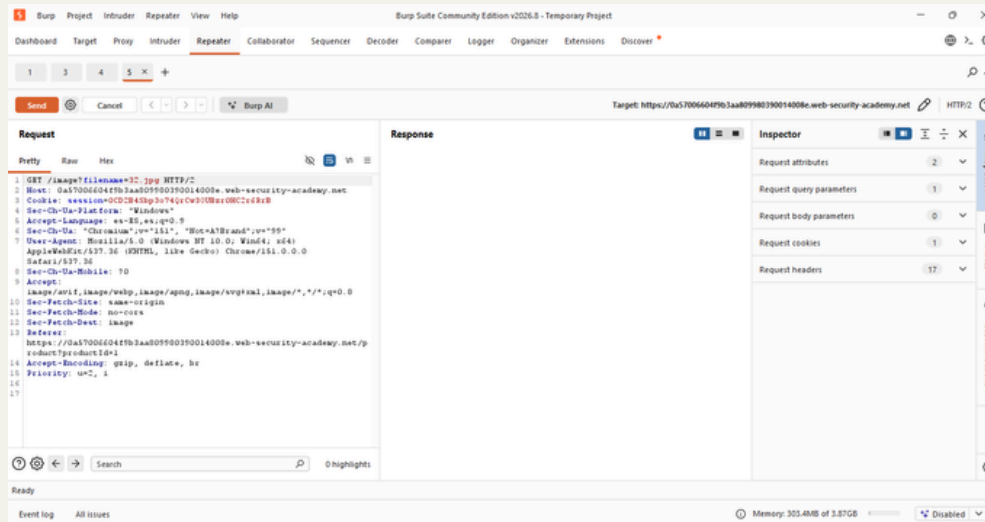


Figura 10.3 Parámetro filename original

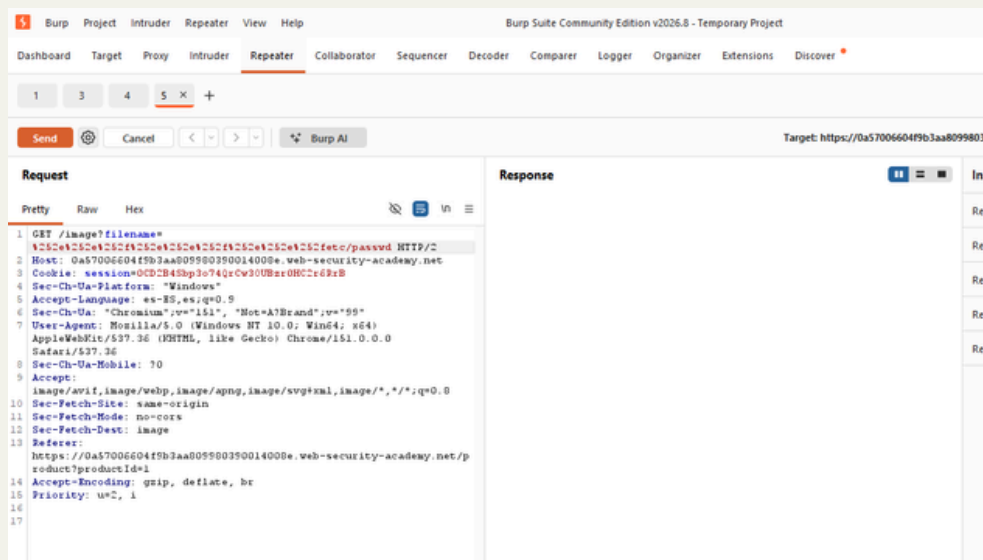


Figura 10.4 Parámetro filename modificado

Paso 4: Se ejecutó la petición maliciosa al servidor. El mecanismo de decodificación redundante de la aplicación tradujo exitosamente la secuencia ofuscada hasta construir la ruta absoluta hacia el archivo objetivo, devolviendo en la respuesta HTTP los datos confidenciales de los usuarios del sistema operativo.

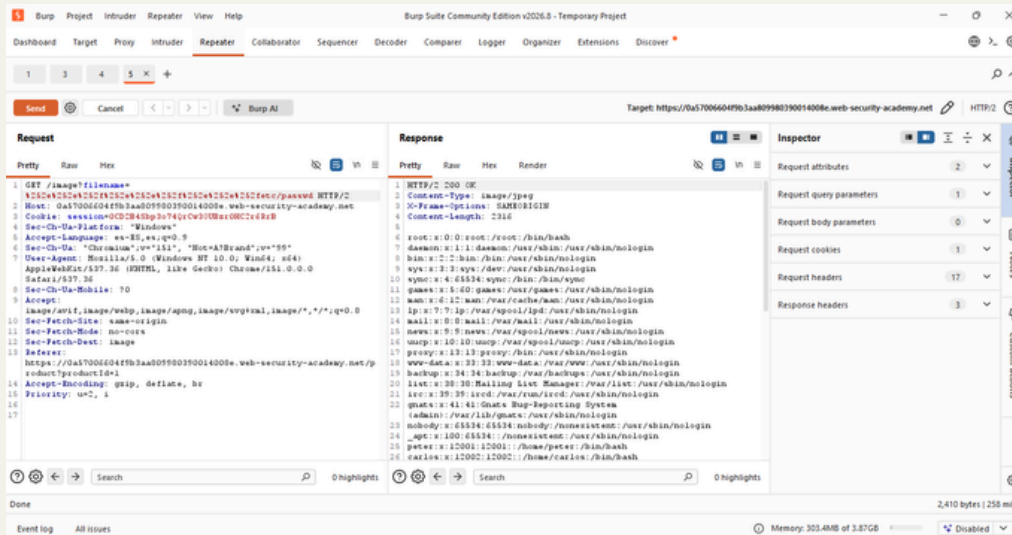


Figura 10.5 Respuesta del servidor al filename modificado

Paso 5: Finalmente, se recargó la vista del navegador, corroborando la correcta explotación de la vulnerabilidad mediante la obtención de la insignia "Lab Solved".

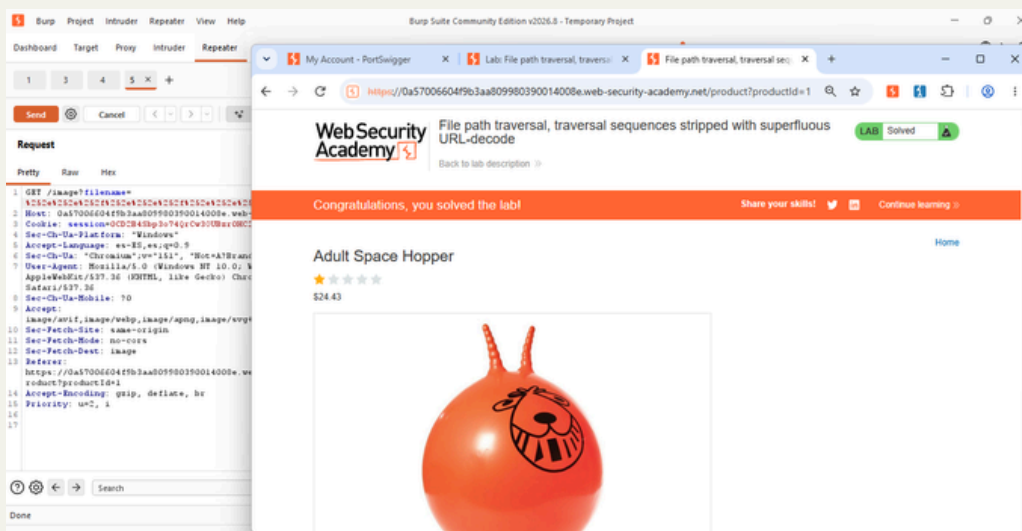


Figura 10.6 Laboratorio finalizado

EXPLICACIÓN DEL PAYLOAD

Payload utilizado: %252e%252e%252f%252e%252e%252f%252e%252e%252fetc/passwd

Explicación técnica: El payload explota el fallo lógico de validación y decodificación. Al enviar %252e%252e%252f, el filtro de validación, que típicamente buscaría un . o un %2e, evalúa la cadena y solo observa caracteres permitidos alfanuméricos y signos de porcentaje que no forman un patrón conocido de salto de directorio.

Inmediatamente después de pasar este filtro, la aplicación implementa de manera errónea una o varias llamadas a funciones de decodificación de URLs (como URLDecoder.decode()). En la primera decodificación, %25 se transforma en %, dejando la cadena como %2e%2e%2f. En la segunda pasada de decodificación (la superflua), %2e%2e%2f se convierte en su forma canónica: ../. Así, el payload se traduce de forma transparente a ../../etc/passwd directamente en la función que lee el archivo.

MITIGACIÓN RECOMENDADA

- Unificar la lógica de decodificación: Garantizar que los datos suministrados por el usuario se decodifiquen de URL estrictamente una sola vez y antes de ejecutar cualquier validación de seguridad. La arquitectura debe evitar decodificaciones implícitas ocultas en diferentes capas del framework (OWASP Foundation, 2023).
- Validación tras la canonicalización: Ejecutar las validaciones de listas blancas (allow-lists) únicamente sobre la forma final del string decodificado y después de resolver la ruta canónica absoluta en el sistema operativo.
- Mapeo indirecto (Indirect Object Reference): Rechazar el paso de rutas de sistema de archivos en los parámetros HTTP. En lugar de procesar nombres de archivos en bruto, el servidor debe recuperar la ubicación real mapeando un ID numérico o hash seguro contra una base de datos de control interno (CISA, 2022).

REFERENCIAS

- CISA. (2022). Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal'). Cybersecurity and Infrastructure Security Agency. <https://www.cisa.gov/uscert/bsi/articles/knowledge/coding-practices/path-traversal>
- MITRE. (2023). CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal'). Common Weakness Enumeration. <https://cwe.mitre.org/data/definitions/22.html>
- OWASP Foundation. (2023). Path Traversal. Open Worldwide Application Security Project. https://owasp.org/www-community/attacks/Path_Traversal